

Khalasni

Service Ordering Website with Admin Control Panel

Project Proposal

Prepared for client review

Document Item	Description
Project Type	Service-ordering website and backend control panel
Recommended Technology	Django, PostgreSQL, Django Admin, Django Templates
Main Scope	Service management, customer orders, payment tracking, and reports
Version	1.0

Confidential proposal document

1. Executive Summary

This proposal describes the development of a web-based service ordering system that allows the business to manage services, receive customer orders, track payment status, and generate operational and payment reports from a centralized control panel.

The system will be built using Django as the backend framework. Django is suitable for this type of project because it provides a reliable backend structure, secure authentication, a built-in admin interface, database management, and future expandability.

The proposed system is intentionally focused on the current business need: service management, order management, payment tracking, and reporting. It does not include unnecessary ERP, accounting, wallet, provider payout, or mobile application complexity at this stage. This keeps the system clean, practical, and expandable.

2. Project Objective

The objective of this project is to build a professional service-ordering website that enables the business to display available services, receive customer orders, manage them from an admin panel, track payment information, and generate reports for operational follow-up.

3. Proposed System Overview

The system will consist of two main parts: a public website for customers and an admin control panel for business users.

Area	Main Purpose	Main Capabilities
Public Website	Allow customers to view services and submit orders.	Service listing, service details, order form, customer information, request notes, and confirmation page.
Admin Control Panel	Allow the business team to manage operations.	Add/edit/remove services, manage categories, view orders, update order status, record payments, and view reports.

4. Core System Modules

4.1 User and Admin Management

This module controls access to the backend system and protects administrative functions.

- Admin login
- Staff user accounts
- Role-based access where needed
- Secure password management
- Permission separation between normal users and administrators

4.2 Service Management Module

This module allows the business to manage the services shown on the website.

- Add service categories
- Add new services
- Edit service name, description, image, and price
- Enable or disable services
- Organize services by category
- Control which services appear on the website

Example Field	Description
Service name	The public name of the service.
Category	The group under which the service is listed.
Description	Detailed explanation shown to customers.
Base price	Optional starting price or fixed price.
Image	Service image displayed on the website.
Active status	Controls whether the service appears publicly.
Display order	Controls service ordering on the website.

4.3 Public Service Display

This module displays the available services to customers through a responsive website.

- Services listing page
- Service category page
- Service details page
- Order button for each service
- Responsive design for desktop and mobile browsers

4.4 Order Management Module

This is the main operational module. It allows customers to submit service requests and allows the admin team to manage them from the backend.

- Customer submits a service order
- Admin receives the order in the control panel
- Admin reviews order details
- Admin updates order status
- Admin adds internal notes
- Admin searches and filters orders

Suggested Order Status	Meaning
New	The order has been submitted and is waiting for review.
Confirmed	The order has been reviewed and accepted.
In Progress	The service is being handled.
Completed	The service has been completed.
Cancelled	The order has been cancelled.

4.5 Payment Tracking Module

This module does not perform full accounting. It only tracks payment-related information for each order and supports reporting.

- Record payment amount
- Record payment method
- Record payment status
- Add payment reference number
- Upload receipt or payment proof if needed
- Track paid and unpaid orders
- Filter payments by date, method, and status

Payment Method	Use
Cash	Payment collected directly.
CliQ	Manual transfer reference can be recorded.
Bank Transfer	Bank transfer reference can be recorded.
Card	For card machine or future gateway use.
Other	Fallback method for special cases.

Payment Status	Meaning
Pending	Payment has not been confirmed yet.
Paid	Full payment has been received.
Partially Paid	Only part of the amount has been received.
Failed	Payment attempt failed or was not completed.
Refunded	Payment was refunded.

4.6 Reporting Module

This module provides the business with operational and payment reports.

- Daily orders report
- Monthly orders report
- Orders by status
- Orders by service
- Paid orders report
- Pending payment report
- Payment by method report
- Revenue/payment summary by date
- Service performance report

The system can support exporting reports to CSV or Excel if required.

4.7 Website Content Management

A simple content management module can be included for basic website information.

- Website name
- Contact phone
- WhatsApp number
- Email
- Address

- Social media links
- Homepage banner
- Basic static content

5. Proposed Technical Architecture

Layer	Recommended Component	Purpose
Backend	Django	Core business logic, models, forms, views, and admin panel.
Database	PostgreSQL	Reliable storage for services, orders, payments, and reports.
Frontend	Django Templates with responsive HTML/CSS	Fast and practical website implementation without unnecessary frontend complexity.
Admin Panel	Django Admin	Control panel for managing services, orders, payments, and website settings.

For this version, Django templates are recommended instead of React or Flutter because the current requirement is a service website and control panel, not a complex interactive mobile application.

6. Recommended Django Project Structure

The project will be organized into separate Django apps to keep the code clean and expandable.

```
apps/  
├── accounts/  
├── services/  
├── orders/  
├── payments/  
├── reports/  
├── cms/  
└── core/
```

App	Responsibility
accounts	Users, admin accounts, and staff permissions.
services	Service categories and services.
orders	Customer service orders and order status.
payments	Payment records and payment status.
reports	Order and payment reports.
cms	Website content, banners, and contact information.
core	Shared base models and utilities.

7. Main Database Entities

The first version can be built with a small and clear database structure.

- User
- ServiceCategory
- Service
- ServiceOrder
- PaymentRecord
- SiteSetting
- Banner

Relationship overview:

ServiceCategory → Service → ServiceOrder → PaymentRecord

8. Main User Flows

Flow	Steps
Customer Order Flow	Customer opens website → browses services → selects service → fills order form → submits order → receives confirmation → admin manages the order.
Admin Service Management Flow	Admin logs in → opens services section → adds or edits service → service appears on website → admin can disable service when needed.
Admin Order Management Flow	Admin logs in → views new orders → confirms order → updates status → records payment information → closes completed order.
Payment Reporting Flow	Admin records payment → payment status updates → payment appears in reports → admin filters by date/status/method → admin exports report if needed.

9. Scope of Work

9.1 Included in the Proposed System

- Django project setup
- Database setup
- Admin/staff login
- Service category management
- Service management
- Public service listing pages
- Service details page
- Customer order form
- Order management in admin panel
- Payment tracking
- Basic payment reports
- Basic order reports
- Website content settings
- Responsive web design
- Deployment preparation
- Basic security configuration
- Basic handover documentation

9. Future Expansion Possibilities

The system will be built in a way that allows future expansion when the business is ready.

- Customer accounts
- Provider accounts
- Provider assignment
- Provider dashboard
- Online payment gateway
- WhatsApp/SMS notifications
- Email notifications
- Advanced reporting
- API layer using Django REST Framework
- Mobile application
- Multi-language support
- Coupon and promotion system
- Reviews and ratings
- Complaint management
- Advanced dashboard

10. Security and Reliability Considerations

- Use secure admin passwords
- Restrict admin panel access to authorized users
- Use HTTPS in production
- Protect uploaded files
- Validate customer input
- Use environment variables for secrets
- Configure database backups
- Limit file upload types and sizes
- Keep dependencies updated
- Separate development and production settings

11. Expected Deliverables

1. Functional service ordering website
2. Admin control panel
3. Service management module
4. Order management module
5. Payment tracking module
6. Basic reporting module
7. Website content settings
8. Database structure
9. Deployment-ready Django project
10. Basic user/admin guide

12. Success Criteria

- Admin can add, edit, disable, and remove services
- Customers can view services on the website
- Customers can submit service orders
- Admin can manage submitted orders
- Admin can update order status
- Admin can record payment information
- Admin can view payment reports
- Admin can view order reports
- The system is stable, structured, and ready for future expansion

13. Final Recommendation

The recommended solution is to build a Django-based service ordering website with an admin control panel.

This solution fits the current scope because it focuses on the actual required business functions: service management, order management, payment tracking, reporting, and website control panel. It avoids unnecessary complexity while keeping the system ready for future growth.